



3

LECTURE 3

Complexity Analysis of Algorithms

CS24 · Fall 2026

Diba Mirza

How to analyze the efficiency of your algorithms

Problem: Fibonacci Numbers

Definition:

The Fibonacci numbers are the sequence

1, 1, 2, 3, 5, 8, 13, 21, 34, 55,...

Defined by

$$F_0 = F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2} \text{ for } n \geq 2$$

Problem: Given n , compute F_n .

Which implementation is significantly faster ?

A.

```
F(int n){  
    if(n <= 1) return 1  
    return F(n-1) + F(n-2)  
}
```

B.

```
F(int n){  
    Initialize A[0 . . . n]  
    A[0] = A[1] = 1  
    for i = 2 : n  
        A[i] = A[i-1] + A[i-2]  
    return A[n]  
}
```

C. *Both are almost equally fast*

Which implementation is significantly faster ?

A.

```
F(int n){  
    if(n <= 1) return 1  
    return F(n-1) + F(n-2)  
}
```

B.

```
F(int n){  
    Initialize A[0 . . . n]  
    A[0] = A[1] = 1  
    for i = 2 : n  
        A[i] = A[i-1] + A[i-2]  
    return A[n]  
}
```

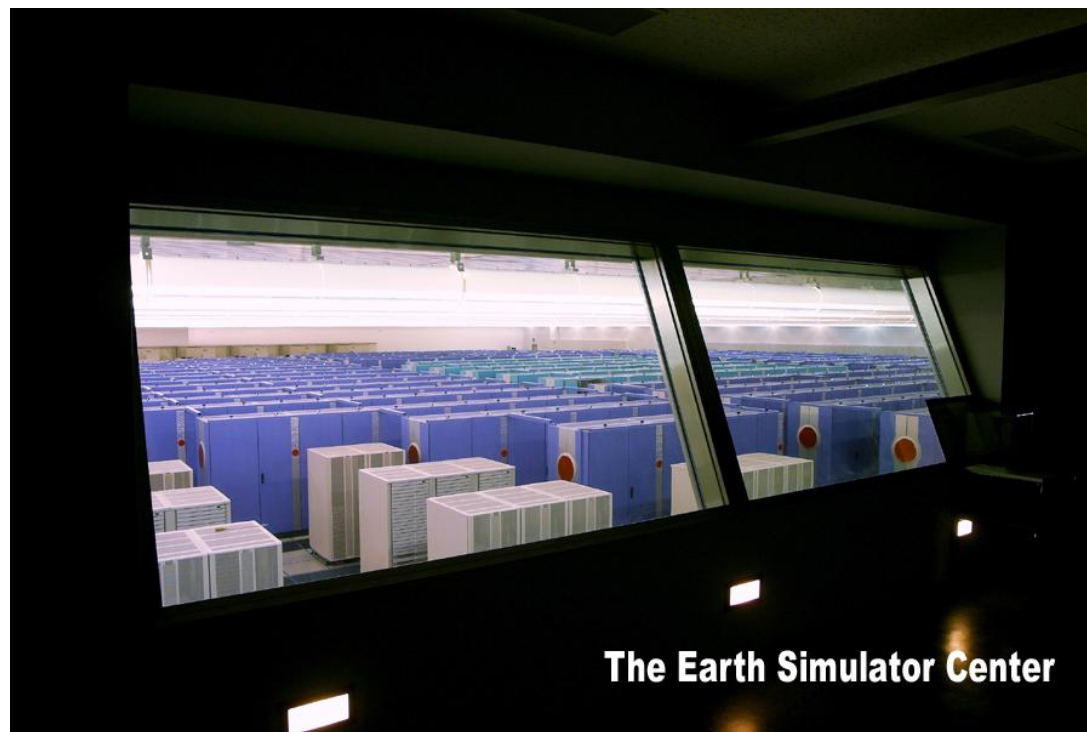
C. *Both are almost equally fast*

The “right” question is: How does the running time grow?

E.g. How long does it take to compute $F(200)$ recursively?

....let's say on....a supercomputer that can compute 40 trillion operations per sec

NEC Earth Simulator



Can perform up to 40 trillion operations per second.

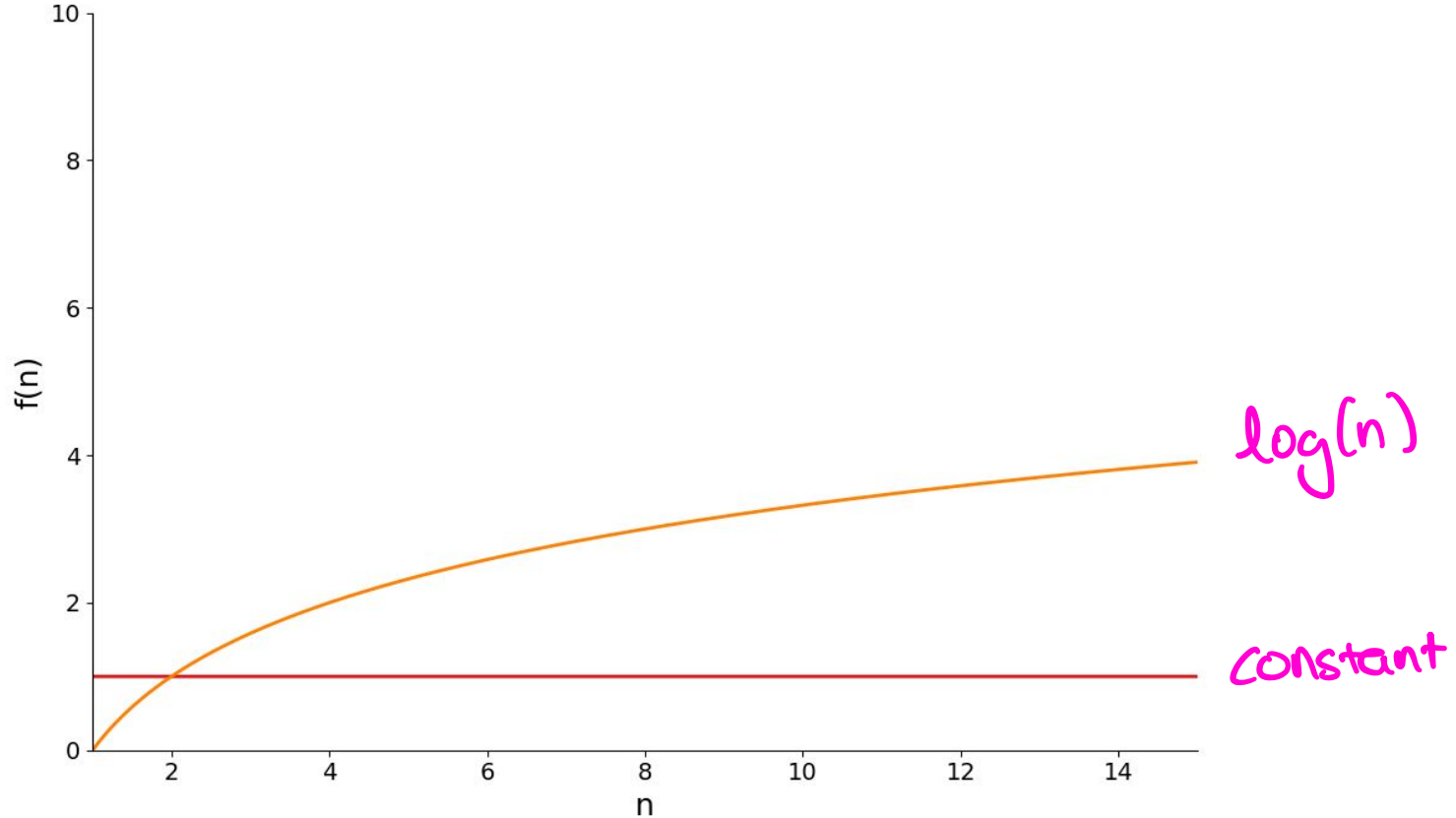
How long does it take to compute $\text{Fib}(200)$ recursively?

....let's say on.... a supercomputer that runs 40 trillion operations per second

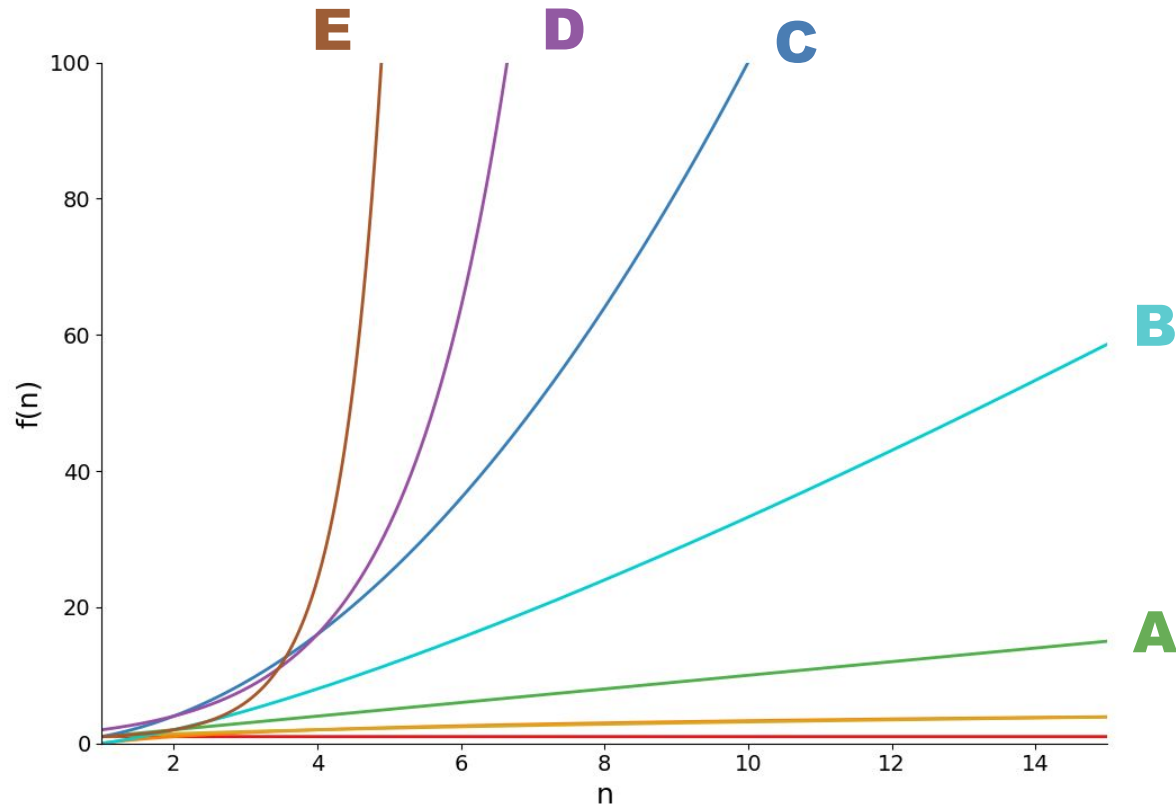
It will take approximately 2^{92} seconds to compute F_{200} .

Time in seconds	Interpretation
2^{10}	17 minutes
2^{20}	12 days
2^{30}	32 years
2^{40}	35000 years (cave paintings)
2^{50}	35 million years ago
2^{70}	Big Bang

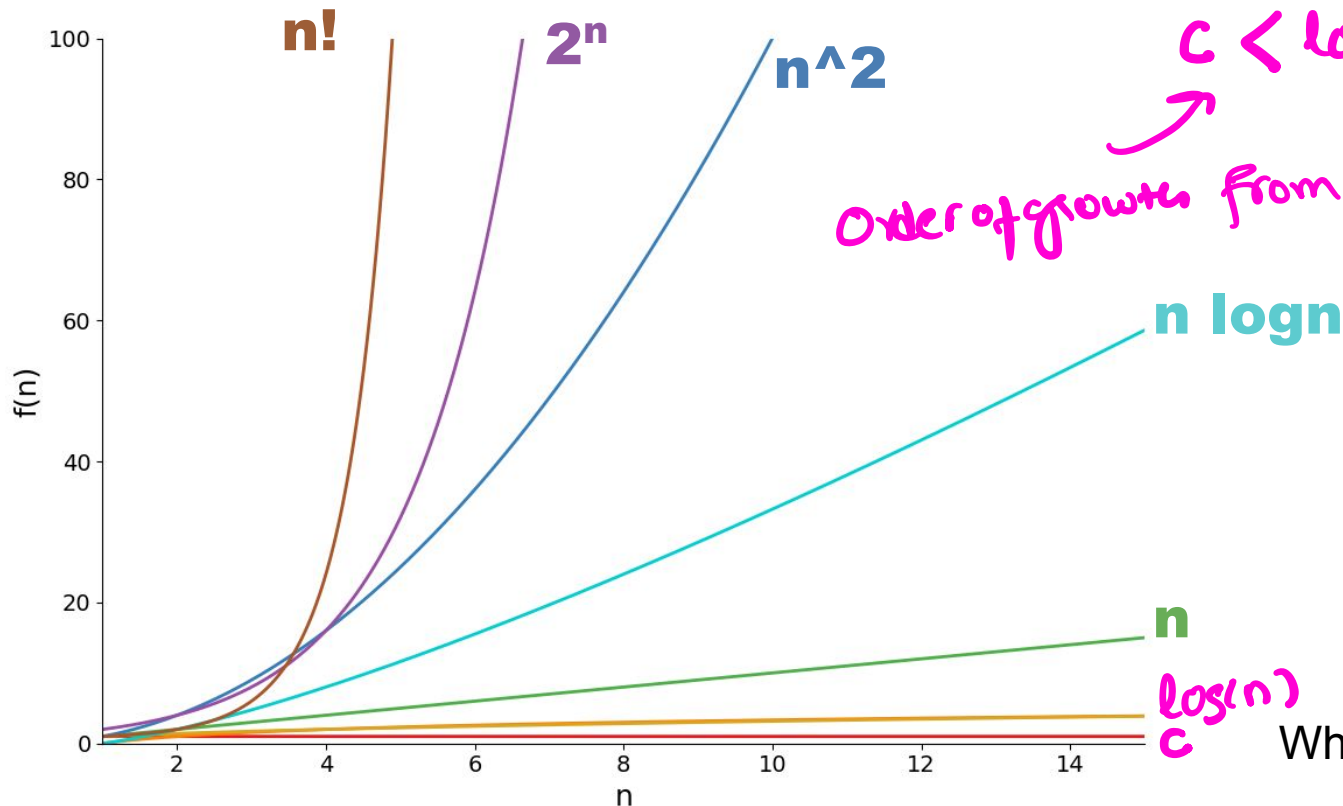
Key idea: Use functions to describe growth (time, space, any other variable of interest)
Identify the two functions in this plot.



Which curve represents how the running time of the recursive fibonacci algo grows?



- An **order of growth** is a set of functions whose growth behavior is considered equivalent.
- Functions that grow similarly belong to the same order of growth



$c < \log n < n < n \log n < n^2 < 2^n < n!$
 Order of growth from "smallest" to "largest"

What about $f(n) = 5n^2 + 10 \log n$?

Based on your reading on Big-O, for $f(n) = 5n^2 + 10 \log n$, which statement is true?

$$f(n) = O(n^2)$$

A. $f(n) = O(\log n)$, because a log grows slower than a square and slower means it dominates

B. $f(n) = O(n^2)$, because we drop the 5 and ignore the log

$$f(n) = O(g(n))$$

C. $f(n) = O(n^2)$: there exist constants $c > 0$ and $k > 0$ such that $f(n) \leq c \cdot n^2$ for every $n \geq k$

Definition of Big-O

$$g(n)$$

D. $f(n) = O(5n^2 + 10 \log n)$ only; Big-O does not allow simplification

E. $f(n)$ is not $O(n^2)$, because of the extra $10 \log n$ term

Big-O Notation

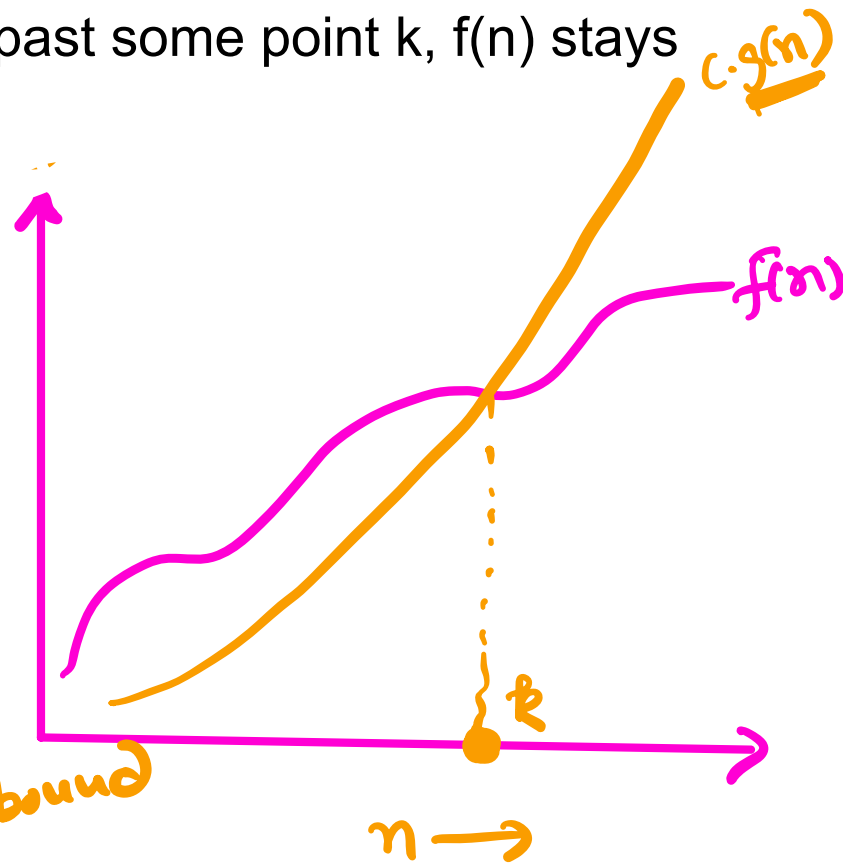
$f(n) = O(g(n))$ means f grows no faster than g : past some point k , $f(n)$ stays under a constant multiple of $g(n)$.

$$f(n) = 5n^2 + 10 \log n$$

$$\leq 5n^2 + 10 \underbrace{\log n}_{\leq n^2}, \quad n \geq 2$$

$$= 15n^2, \quad n \geq 2 \quad \text{--- } k$$

$$\overset{C}{\rightarrow} = O(n^2)$$



~~$O(2^n)$~~ $\rightarrow$ In practice, choose the smallest upper bound

Express in Big-O notation

$$f(n) = O(g(n))$$

↑
overloaded o perche

1. 10000000 $\rightarrow O(1)$
2. $3n$
3. $6n-2 \rightarrow O(n)$
4. $15n + 44$
5. $50n \log(n) \rightarrow O(n \log n)$
6. n^2
7. $n^2 - 6n + 9$
8. $3n^2 + 4 \log(n) + 1000$
9. $3^n + n^3 + \log(3 \cdot n) \rightarrow O(\underline{3^n})$
 ↑ exp. ↑ polynomial

Common sense rules

1. Multiplicative constants can be omitted: $14n^2$ becomes n^2 .
2. n^a dominates n^b if $a > b$: for instance, n^2 dominates n .
3. Any exponential dominates any polynomial: 3^n dominates n^5 (it even dominates 2^n).

For polynomials, use only leading term, ignore coefficients: linear, quadratic

Running time Big O analysis: Iterative Fibonacci

Lets compute $T(n)$ = number of primitive operations to execute $F(n)$

$$T(n) = O(n)$$

e.g. add/subtract/compute /
load/store
Basic operations any computer supports

```
F(int n){
```

```
  + Initialize A[0 . . . n] →  $O(n)$  or  $C \cdot n$ 
```

```
  + A[0] = A[1] = 1  $O(1)$ 
```

```
  +
```

```
    for (int i = 2; i <= n; i++) {
```

```
  + A[i] = A[i-1] + A[i-2] → constant time
```

```
  + return A[n]  $O(1)$ 
```

```
}
```

loop runs

$n-1$ times

Alternatively,

$$T(n) = O(n) + O(1) + (n-1)O(1) + O(1)$$

$$= O(n)$$

$$T(n) = C \cdot n + C_1 + C \cdot (n-1) + C_2$$

$$= O(n) \quad (\text{using the simplification from previous slide})$$

Big O running time analysis: clicker

```

/* n is the length of the array*/
int sum(int arr[], int n)
{
    int result = 0;
    for(int i = 0; i < n; i+=2)
        result+=arr[i];
    return result;
}

```

$n/2$ times

$$T(n) = C_1 + C_2 \cdot \frac{n}{2} = O(n)$$

- A. $O(n^2)$
- ☒ B. $O(n)$
- C. $O(n/2)$
- D. $O(\log n)$
- E. None of the above

Why So Slow?

```
F(int n){  
    if(n <= 1) return 1  
    return F(n-1) + F(n-2)  
}
```

$T(n) = c \cdot (\# \text{ of function calls})$

Show $T(n) = O(2^n)$

running time



Level

of function calls
per level

0

1

1

2

2

4

3

8

4

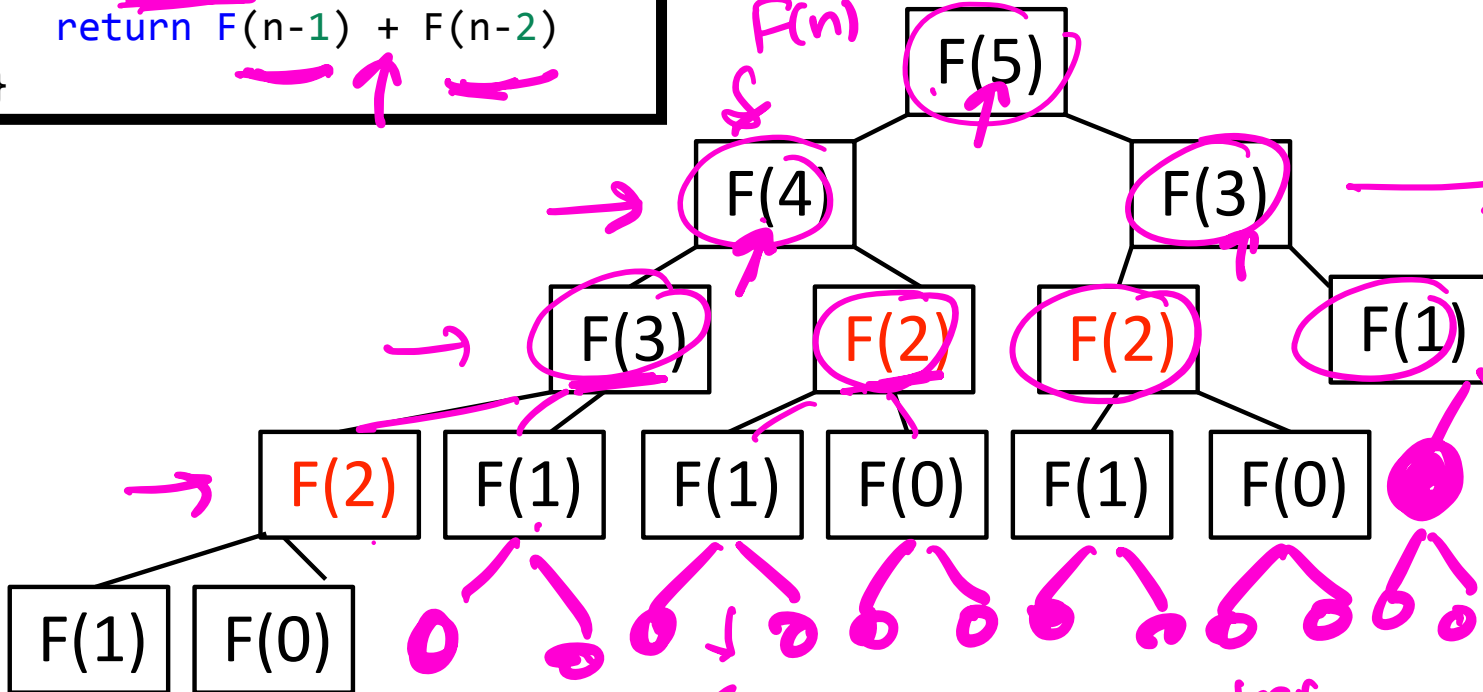
16

...

...

(n-1)

2



1 + 2 + 4 + 6 + 2 ← No clear pattern!

Sum to get the total

Show $T(n) = O(2^n)$

$T(n) = c \cdot \# \text{ of function calls}$

$$\leq c \cdot (2^0 + 2^1 + 2^2 + \dots + 2^{n-1})$$

$$= c \cdot (2^n - 1) \quad (\text{Sum of geometric series, known result})$$

$$= O(2^n)$$

Because we augmented the call tree
(on the previous page)

```
F(int n){  
    if(n <= 1) return 1  
    return F(n-1) + F(n-2)  
}
```

What is the largest 1-byte number?

$$\begin{array}{r} 11111111 \\ \hline 2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 \\ = 2^0 + 2^1 + \dots + 2^7 \\ = 255 \\ = 2^8 - 1 \end{array}$$

$$\text{In general, } 2^0 + 2^1 + \dots + 2^k = 2^{k+1} - 1$$

Space Complexity

extra space

$S(n)$ = auxiliary memory needed to compute $F(n)$

Auxiliary space: Peak space usage not considering input and output

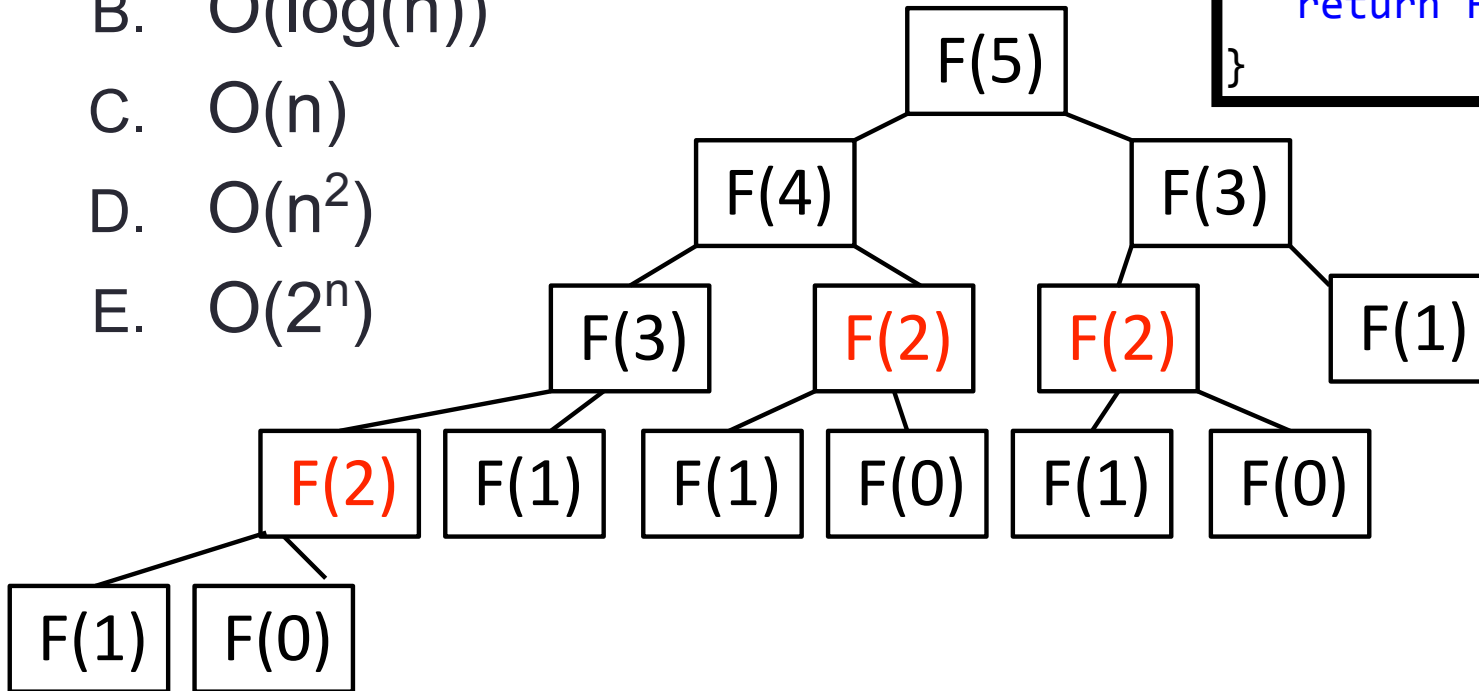
In the case of the iterative fibonacci, we used an array of size $n+1$ to compute $Fib(n)$. Therefore $S(n)$ for iterative fibonacci is $O(n)$. We write this as. $S(n) = O(n)$

What is $S(n)$? Express your answer in Big-O notation

What is $S(n)$? Express your answer in Big-O notation

- A. $O(1)$
- B. $O(\log(n))$
- C. $O(n)$
- D. $O(n^2)$
- E. $O(2^n)$

```
F(int n){  
    if(n <= 1) return 1  
    return F(n-1) + F(n-2)  
}
```



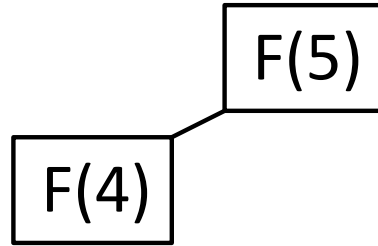
Tree of recursive calls needed to compute $F(5)$

$S(n)$ relates to maximum depth of the recursion

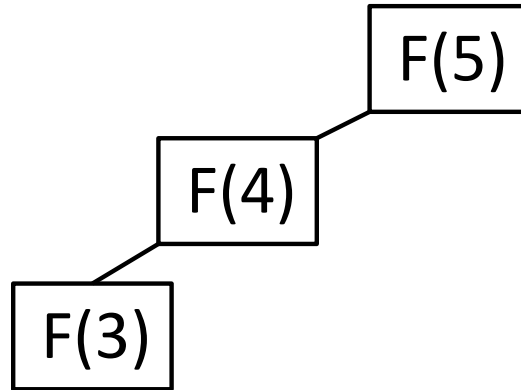
```
F(int n){  
    if(n <= 1) return 1  
    return F(n-1) + F(n-2)  
}
```

F(5)

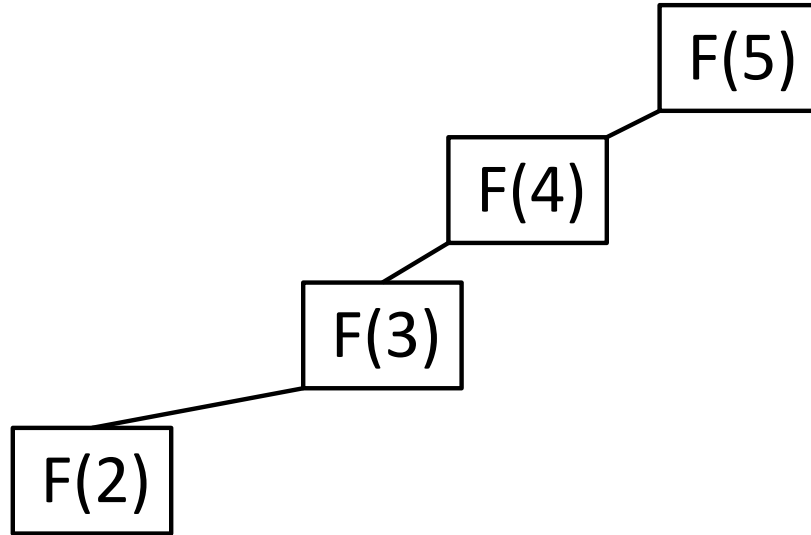
$S(n)$ relates to maximum depth of the recursion



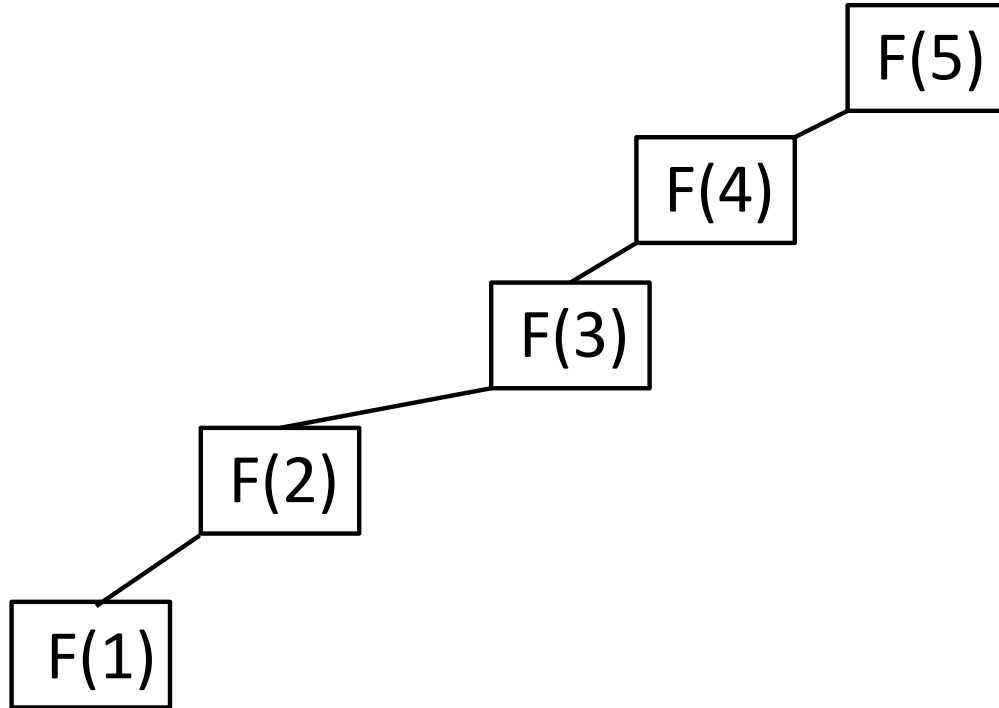
$S(n)$ relates to maximum depth of the recursion



$S(n)$ relates to maximum depth of the recursion

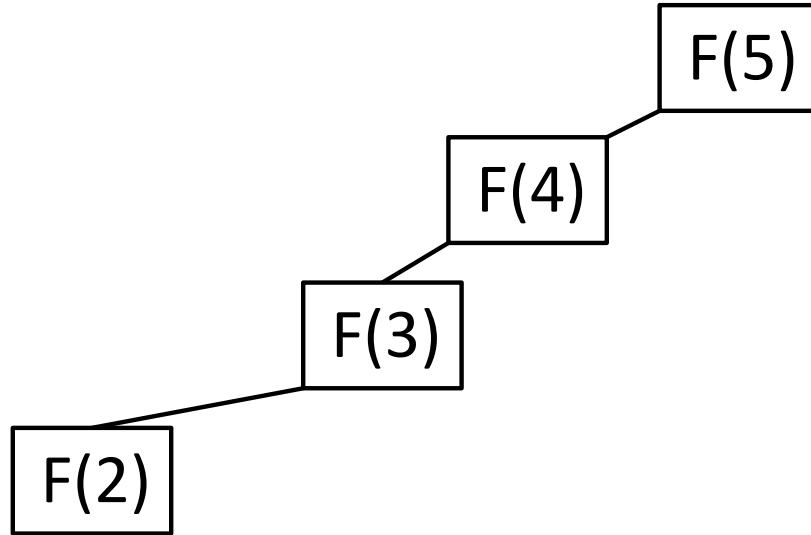


$S(n)$ relates to maximum depth of the recursion



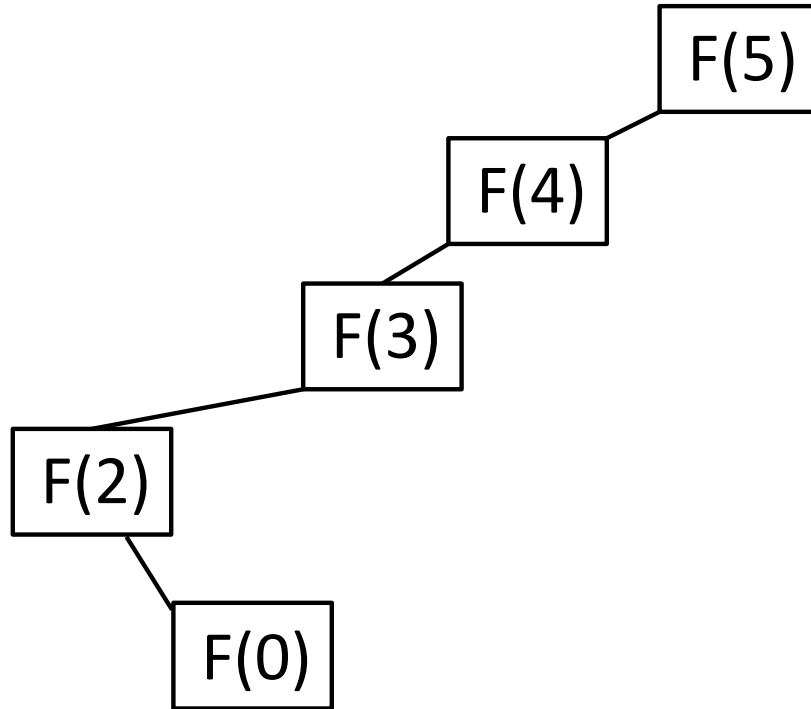
Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



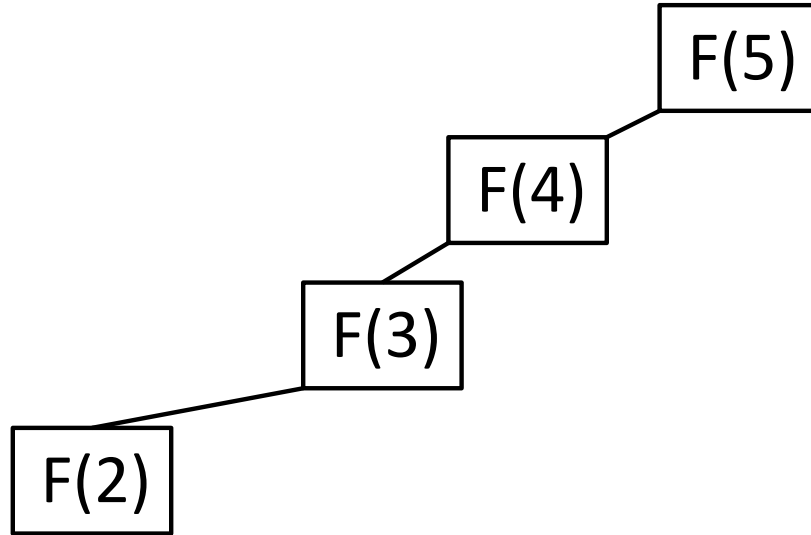
Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



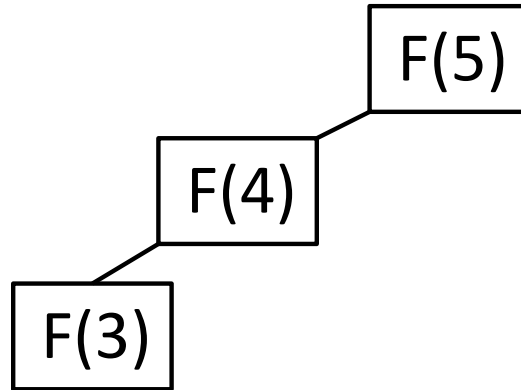
Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



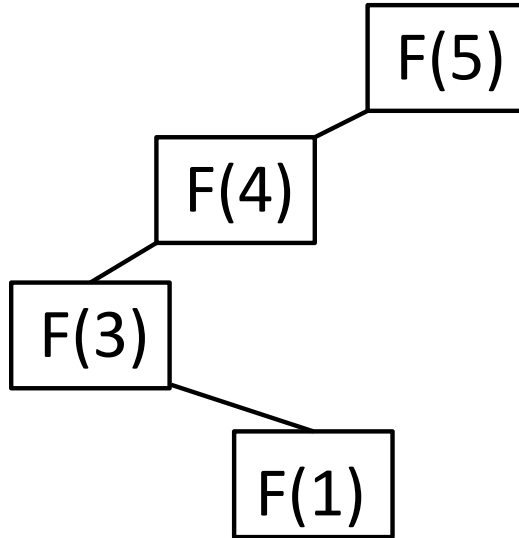
Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



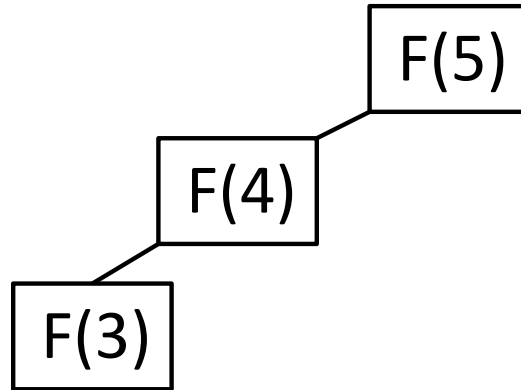
Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



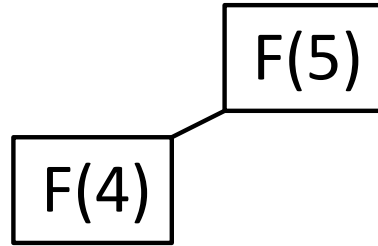
Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



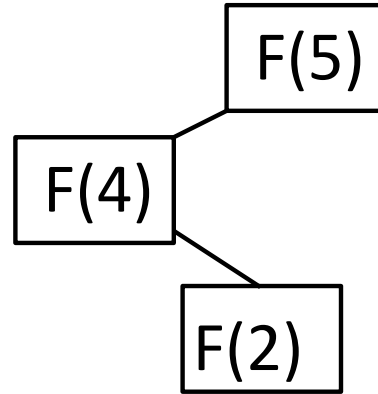
Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



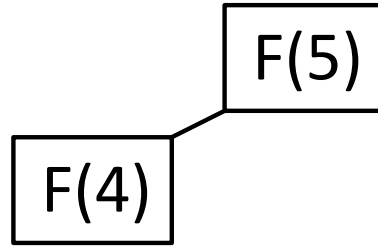
Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



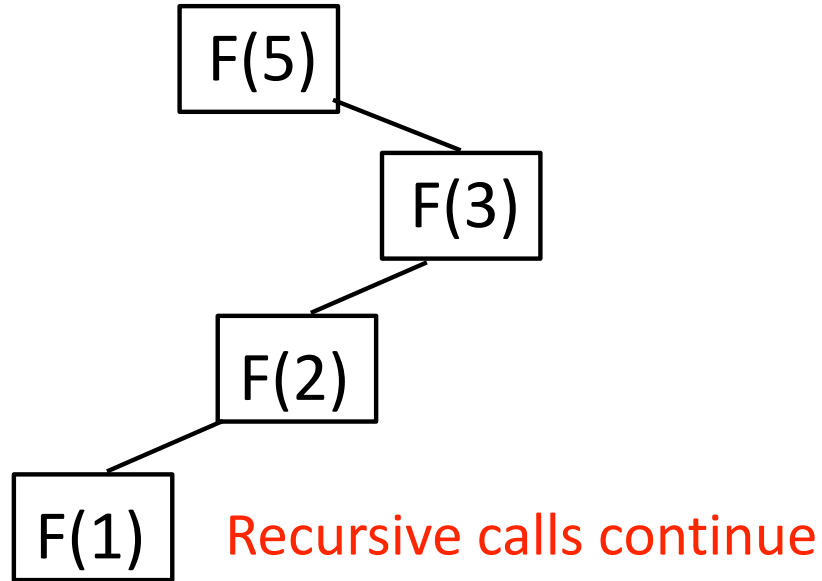
Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



Maximum depth of the recursion = 5

$S(n)$ relates to maximum depth of the recursion



Maximum depth of the recursion for $F(n) = n$

Therefore, $S(n) = O(n)$

Which algorithm is more space efficient?

A.

```
F(int n){  
    if(n <= 1) return 1  
    return F(n-1) + F(n-2)  
}
```

B.

```
F(int n){  
    Initialize A[0 . . . n]  
    A[0] = A[1] = 1  
    for i = 2 : n  
        A[i] = A[i-1] + A[i-2]  
    return A[n]  
}
```

C. Both are the same: $O(n)$

Which of these questions can we answer using Big-O?

- A. Why is Algo A so slow?
- B. Is Algo A better than Algo B?
- C. When will my code finish running?

Next time

- Quiz 1: Includes Lecture 1 to 3 (up to rule of three).
- 35 minutes during lecture
- Bring dark pencil or pen
- Binary Search Trees

Credits and references:

Slides based on presentations by Professors Sanjoy Das Gupta and Daniel Kane at UCSD
<https://cseweb.ucsd.edu/~dasgupta/book/toc.pdf>